



Introdução ao Kimi via API

Este notebook apresenta recursos iniciais do **Kimi** (Moonshot AI) consumidos via API, mostrando o quão poderoso pode ser o modelo em tarefas de linguagem, visão, ferramentas e geração estruturada.

Modelo usado: `kimi-k3` (ou `kimi-k2-0711-preview`, conforme disponibilidade)

Documentação: <https://platform.moonshot.cn/docs>

1. Instalação e configuração

```
# Instale a biblioteca oficial (OpenAI-compatible)
!pip install -q openai python-dotenv
```

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

client = OpenAI(
    api_key=os.getenv("KIMI_API_KEY"),
    base_url="https://api.moonshot.ai/v1"
)

MODEL = "kimi-k3"
```

2. Chat completion simples

O básico: envie uma lista de mensagens e receba uma resposta.

3. Respostas em streaming

O streaming permite mostrar texto aos poucos, ideal para interfaces interativas.

```
stream = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "Escreva um parágrafo sobre inteligência artificial generativa."}],
    stream=True,
)

for chunk in stream:
    content = chunk.choices[0].delta.content
    if content:
        print(content, end="", flush=True)
```

4. System prompt e controle de comportamento

O system prompt define a persona, formato e restrições do modelo.

```
system_prompt = """
Você é um tradutor técnico sênior. Regras:
1. Traduza do português para o inglês técnico.
2. Mantenha termos de código e APIs em inglês.
```

3. Responda APENAS com a tradução, sem explicações.

"""

```
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": "O método __init__ é o construtor da classe e recebe self como primeiro argumento."}
]

print(chat(messages))
```

5. Function Calling (tools)

O modelo pode decidir chamar funções externas para obter dados em tempo real. Aqui simulamos uma consulta de previsão do tempo.

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Obtém a previsão do tempo para uma cidade.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "Nome da cidade"},
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "Unidade de temperatura"
                    }
                }
            },
            "required": ["city"]
        }
    }
]

messages = [
    {"role": "user", "content": "Qual a temperatura em São Paulo agora?"}
]

response = client.chat.completions.create(
    model=MODEL,
    messages=messages,
    tools=tools,
    tool_choice="auto"
)

message = response.choices[0].message
print(message)
```

```
# Simula o resultado da função e devolve ao modelo
def get_weather(city, unit="celsius"):
    return {"city": city, "temperature": 22, "unit": unit, "condition": "parcialmente nublado"}

if message.tool_calls:
    tool_call = message.tool_calls[0]
    function_name = tool_call.function.name
    arguments = eval(tool_call.function.arguments)  # em produção, use json.loads

    if function_name == "get_weather":
        result = get_weather(**arguments)

    messages.append({
        "role": "assistant",
```

```

        "content": None,
        "tool_calls": [tool_call.model_dump()]
    })
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": str(result)
    })

    final_response = client.chat.completions.create(
        model=MODEL,
        messages=messages
    )
    print(final_response.choices[0].message.content)

```

6. JSON Mode / saída estruturada

Force o modelo a responder em JSON válido, útil para integração com código.

```

import json

messages = [
    {"role": "system", "content": "Você é um analisador de produtos. Responda SEMPRE em JSON válido."},
    {
        "role": "user",
        "content": ""
    }
]

Extraia os pontos positivos e negativos do produto descrito abaixo.

Produto: Smartphone KimiPhone Pro
"A câmera é excelente em pouca luz, mas a bateria dura pouco mais de meio dia com uso intenso. O
desempenho é fluido e a tela tem cores vibrantes."

Formato esperado:
{
    "produto": "string",
    "pontos_positivos": ["string"],
    "pontos_negativos": ["string"],
    "nota_geral": number
}
"""
}

]

response = client.chat.completions.create(
    model=MODEL,
    messages=messages,
    response_format={"type": "json_object"}
)

data = json.loads(response.choices[0].message.content)
print(json.dumps(data, indent=2, ensure_ascii=False))

```

8. Conversa multi-turn com memória

Mantenha o histórico de mensagens para criar diálogos coerentes.

```

conversation = [
    {"role": "system", "content": "Você é um tutor de programação paciente."}
]

def ask(question):
    conversation.append({"role": "user", "content": question})
    answer = chat(conversation)
    conversation.append({"role": "assistant", "content": answer})
    return answer

```

```
print("Turno 1:", ask("O que é uma API REST?"))
print("\nTurno 2:", ask("E qual a diferença para GraphQL?"))
print("\nTurno 3:", ask("Me dê um exemplo de quando escolher cada uma?"))
```

9. Chain-of-Thought: resolvendo problemas passo a passo

Incentive o modelo a mostrar o raciocínio antes da resposta final.

```
messages = [
    {
        "role": "user",
        "content": """
Resolva passo a passo:
Um trem parte de A para B a 60 km/h. Outro parte de B para A a 80 km/h.
A distância entre A e B é 280 km. Em quanto tempo eles se encontram?
Mostre o raciocínio completo antes da resposta final.
"""
    }
]

print(chat(messages, temperature=1.0))
```

11. Análise de tokens e custo estimado

A API retorna informações de uso, permitindo estimar custos.

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "Resuma a Revolução Industrial em 3 frases."}]
)

usage = response.usage
print(f"Prompt tokens: {usage.prompt_tokens}")
print(f"Completion tokens: {usage.completion_tokens}")
print(f"Total tokens: {usage.total_tokens}")
```

12. Requisições assíncronas (opcional)

Use a versão assíncrona do cliente para chamadas concorrentes.

```
import asyncio
from openai import AsyncOpenAI

async_client = AsyncOpenAI(
    api_key=os.getenv("KIMI_API_KEY"),
    base_url="https://api.moonshot.ai/v1"
)

async def perguntar(assunto):
    r = await async_client.chat.completions.create(
        model=MODEL,
        messages=[{"role": "user", "content": f"Explique {assunto} em uma frase."}]
    )
    return r.choices[0].message.content

resultados = await asyncio.gather(
    perguntar("machine learning"),
    perguntar("blockchain"),
    perguntar("computação em nuvem")
)
```

```
for r in resultados:
    print("-", r)
```

Próximos passos

- Experimente prompts mais complexos para seu domínio.
- Combine function calling com JSON mode para agents.
- Implemente RAG (Retrieval-Augmented Generation) usando embeddings.
- Avalie diferentes valores de `temperature` e `top_p`.
- Consulte a documentação oficial para limites de contexto e preços.